RabbitTrack — Database Structure & Data Review

A complete review of how the application stores its data: the tables, their identifiers, the relationships between them, the offline mirror, and an analysis of where data is (and isn't) duplicated.

Application	RabbitTrack — rabbit breeding-farm manager
Report date	24 July 2026
Server database	PostgreSQL (Neon) via Prisma 7
Offline database	SQLite mirror (Capacitor / Electron client)
Server models	22 tables
Mobile mirror tables	17 tables

Core herd · Live breeding cycle · Permanent archive logs · Financial & stock · Sync & accounts

Contents

1 System architecture — two databases, one truth

RabbitTrack keeps data in **two** places, by design:

- **PostgreSQL (server, the source of truth).** The Next.js web app writes to it directly through Prisma. Every farm's data lives here.
- **SQLite (on each phone/tablet/desktop client).** A local *mirror* so the app works fully offline. Clients never talk to Postgres directly — they *pull* rows down and *push* queued operations up.

The two are reconciled by a sync engine: a device downloads changes since its last cursor (*pull*), and uploads business actions it performed offline (*push*) as an idempotent operation log. This is why several design choices below (client-generated IDs, append-only logs, snapshots) exist — they make the same record survive a round-trip through an unreliable network without duplicating or losing history.

> **Money & weight are integers, never floats**
>
> Money is stored as integer **cents** (`amountCents`) and weight as integer **grams** (`weightGrams`). This avoids rounding drift in financial and growth calculations. Dates are stored in UTC and displayed in local time.

2 Multi-tenancy — the `farmId` model

The database is multi-tenant: one server instance can host many farms. The isolation rule is simple and enforced in one place:

- Every farm-owned table carries a `farmId` column pointing at `Farm` .
- Application code never writes `farmId` by hand. A Prisma extension injects the current farm's ID on every create, and adds a `farmId` filter to every read/update/delete automatically.
- The column defaults to an empty string `""` , which matches *no* farm — so a bug that ever skipped the injection fails loudly instead of silently leaking one farm's data into another.
- Deleting a `Farm` cascades through everything it owns.

Accounts and farms are separate: a `User` logs in; `FarmMember` ties a user to a farm with a role (`owner` / `worker`). One account can own one farm and work on another.

3 Identifiers — how every record gets its ID

ID type	Used by	How it's generated & why
`cuid()` primary key	Almost every table's `id`	Collision-resistant, generated by Prisma at insert. Because IDs are globally unique (not auto-increment integers), a row created offline on a phone and a row created on the web can never collide.
Client-generated `id`	Rows born from offline "creating" operations (new breeding, pregnancy-test log, resorption log, etc.)	The *device* mints the ID at the moment of action, so the optimistic local row and the server row that later comes back share one ID. Sync then de-duplicates by ID (`INSERT OR REPLACE`) instead of guessing by date — this is the backbone of duplicate-prevention.
`clientOpId`	`SyncOperation`	A unique ID per queued action. On push, the server looks it up first; a resent batch skips already-applied ops instead of re-running them (idempotency).
Natural keys / uniqueness	See table below	Composite `@@unique` constraints guard against logical duplicates (e.g. one tag number per sex per farm).

Uniqueness constraints in force

Table	Unique on	Meaning
Rabbit	`(farmId, tagId, sex)`	A tag/cage number is unique per sex per farm; many NULL tags allowed (untagged juveniles & deceased freed numbers).
Litter	`breedingId`	Exactly one litter per mating.
Breed	`(farmId, name)`	No duplicate breed names within a farm.
Settings	`farmId` (is the PK)	One settings row per farm.
User	`email`	One account per email.

FarmMember	`(farmId, userId)`	A user joins a farm once.
KitStockMovement	`transactionId`, `rabbitId`	A stock movement mirrors at most one transaction / one retained rabbit.
DeviceToken	`tokenHash`	Login tokens stored only as hashes.
SyncOperation	`clientOpId`	Idempotency — see above.

4 Full table catalog (server / PostgreSQL)

22 tables, grouped by role. **PK** = primary key, → = foreign key.

Core herd

Table	Stores	Key columns & relationships
Rabbit	Every rabbit ever on the farm. Soft-deleted via `status` (never hard-deleted) to preserve pedigree.	**id**; `tagId` (number, freed on death via `retiredTagId`), `sex`, `breed`, `color`, `status`, `doeState`, `origin`, `cage`, `movedToHerdPen`, weights via relation. → sireId, damId (self, pedigree), → litterId (birth litter).
Breed	Curated breed-name menu for dropdowns.	**id**, `name`. Note: `Rabbit.breed` stores the name as free text, *not* a foreign key — so an unregistered breed string never breaks.
Settings	Per-farm display units & breeding-cycle configuration (gestation days, weaning offset, etc.).	**farmId**. Holds `dataResetAt` — the timestamp of the last "wipe online database", which every device checks to detect a server reset.

Live breeding cycle

Table	Stores	Key columns & relationships
Breeding	The *current* breeding cycle of a doe. Reused/overwritten on her next mating (see §7).	**id**; `matingDate` (nullable — cleared on failed mating), `expectedKindlingDate`, `actualKindlingDate`, `nestBoxDate`, `palpationConfirmedDate`, `outcome`, `pregnancyTestResult`. → doeId (required), → buckId (optional).
Litter	The outcome of a successful mating — counts first, individual kits optional.	**id**; → breedingId (unique, 1:1). `kindlingDate`, `bornAlive`, `bornDead`, `weaned`, `weaningDate`, `weaningWeightGrams`. Tagged offspring link back via `Rabbit.litterId`.

Permanent archive logs **(append-only — written once, never edited or deleted by users)**

Table	Stores (one row per…)	Snapshotted columns
MatingLog	…mating event, forever.	**id**, doeId, buckId?, `matingDate`, `wasNursingAtMating`.
PregnancyTestLog	…pregnancy test (عشار / سالبة).	**id**, doeId, buckId?, `matingDate`, `testDate`, `result`.
ResorptionLog	…resorption event (امتصاص).	**id**, doeId, buckId?, `matingDate`, `resorptionDate`.
KindlingLog	…birth (ولادة).	**id**, doeId, buckId?, breedingId?, `matingDate?`, `kindlingDate`, `bornAlive`, `bornDead`.
WeaningLog	…weaning (فطام).	**id**, doeId, buckId?, breedingId?, `kindlingDate?`, `weaningDate`, born counts, `weaned`, `weaningWeightGrams`.
FosterLog	…kit transfer between does (تبني).	**id**, fromDoeId, toDoeId, `count`, `date`.

These exist because the `Breeding` / `Litter` rows they came from get reused on the doe's next cycle. The logs snapshot the facts at the moment of the event, so history never shrinks. This is the single most important structural idea in the schema — explained in §7.

Financial & stock

Table	Stores	Key columns & relationships
Transaction	Income / expense ledger. `rabbitId` null = farm-wide (feed, vet, equipment).	**id**, `date`, `type`, `category`, `amountCents` (always positive; sign implied by type). → rabbitId?.
KitStockMovement	Batch ledger of weaned-kit stock *deductions*: sales, post-weaning deaths, kits retained for breeding.	**id**, `type` (sale/death/retained), `count`, sale `weightGrams` / `pricePerKgCents` / `amountCents`. → transactionId? (mirrors a sale), → rabbitId? (a retained kit). Positive "weaned" totals are *not* stored — they're derived on read from `Litter.weaned`.
WeightRecord	Weight measurements over time (grams).	**id**, → rabbitId, `date`, `weightGrams`. One of the few tables actually mutated in place (has a live `updatedAt`).
HealthRecord	Health events; `nextDueDate` powers the dashboard reminders.	**id**, → rabbitId, `type`, `description`, `date`, `nextDueDate?`.

Sync & accounts

Table	Purpose
Farm	The tenant. Everything else cascades from it.
User	An account (email + scrypt password hash).
FarmMember	User↔Farm membership + role (owner/worker) + per-page permissions.
DeviceToken	Login tokens, stored only as SHA-256 hashes — a DB leak can't be replayed as logins.
SyncDevice	A phone/tablet participating in sync; tracks `lastSyncAt`.
SyncOperation	Idempotency + audit ledger of replayed offline operations, keyed by `clientOpId`.
SyncTombstone	One row per hard-deleted record, so deletions reach every device (a "gone" row can't appear in an `updatedAt > since` diff).

5 Relationships map

The herd table (`Rabbit`) is the hub. A doe flows through a cycle; each stage drops a permanent log row:

```
Farm (tenant)
 └── owns every row below (farmId), cascade-deletes them

Rabbit ──┬── sireId / damId ──▶ Rabbit         (pedigree, self-relation)
         ├── litterId ────────▶ Litter         (which litter it was born in)
         │
         │   as DOE                 as BUCK
         ├── Breeding.doeId         Breeding.buckId          (current cycle)
         ├── MatingLog.doeId        MatingLog.buckId         ┐
         ├── PregnancyTestLog       PregnancyTestLog         │ permanent
         ├── ResorptionLog          ResorptionLog            │ per-event
         ├── KindlingLog            KindlingLog              │ archive
         ├── WeaningLog             WeaningLog               │ logs
         ├── FosterLog.fromDoeId /  toDoeId                  ┘
         │
         ├── WeightRecord, HealthRecord, Transaction         (per-rabbit logs)
         └── KitStockMovement.rabbitId  (a "retained" kit)

Breeding ──1:1──▶ Litter ──▶ Rabbit[] (kits promoted from it)
Transaction ◀──1:1── KitStockMovement (a sale mirrors into the ledger)
```

Cascade on Farm delete	All farm-owned tables (Rabbit, Breeding, logs, Transaction, …).
Cascade on Breeding delete	Its Litter.
Restrict	Can't delete a Rabbit that's referenced as a doe/buck in Breeding or the logs.
SetNull	Pedigree links (sire/dam), `litterId`, and a Transaction's `rabbitId` null out rather than block.

6 The offline mirror (mobile SQLite) & sync tables

The client keeps **17** SQLite tables. Most mirror a Postgres table 1:1 (same column names, so a pulled row stores with no mapping). Differences worth knowing:

- **No `farmId` columns.** A device is bound to exactly one farm, so the mirror is single-tenant and drops the column entirely.
- **Dates are ISO-8601 text; booleans are 0/1.** SQLite has no native datetime/boolean type.
- **Server-only tables are absent** — no User/Farm/FarmMember/DeviceToken/SyncDevice/SyncOperation/SyncTombstone. The device replaces them with two local tables: `outbox` (queued operations) and `sync_cursor` (device identity + pull cursor + last-seen reset timestamp).
- `settings_cache` is a one-row subset of Settings, refreshed every pull.

How each table syncs

Sync strategy	Tables
Incremental (`updatedAt/createdAt > cursor`)	Rabbit, Breeding, Litter, WeightRecord, FosterLog, KitStockMovement, HealthRecord, Transaction
Full recent window (latest 100, re-sent every pull)	MatingLog, PregnancyTestLog, KindlingLog, WeaningLog — so a server-side correction re-propagates.
Whole table every pull	Breed (client fully re-derives it), Settings (single row).
Deletions	SyncTombstone reports hard-deletes for the incremental tables.

> **Minor note — mobile default values differ from the server**
>
> The `settings_cache` table's built-in defaults (`gestationDays 30` , `currency EGP`) differ from the server's Prisma defaults (`31` , `USD`). This is harmless in practice — the row is overwritten by the real server values on the very first pull — but the two default sets could be aligned to avoid confusion for anyone reading the schema cold.

7 The "reused row + permanent log" design

This is the concept that explains most of the schema and is central to the duplication question, so it's worth stating plainly.

A doe has **one** `Breeding` row that represents her *current* cycle. When she is mated again, that same row is **reused**: its `matingDate`, `actualKindlingDate`, `nestBoxDate`, pregnancy result, etc. are reset/overwritten for the new cycle. The `Breeding` row therefore only ever reflects "now", not history.

To keep history, each milestone writes an **append-only log row** (MatingLog, PregnancyTestLog, ResorptionLog, KindlingLog, WeaningLog, FosterLog) that *snapshots* the facts — doe, buck, mating date, counts — at that instant. Those rows are never edited or deleted by users; they are cleared only by an explicit "wipe/reset online database" action (which bumps `dataResetAt` and forces every device to re-bootstrap).

> **Consequence for reports**
>
> Anything that must count history correctly — fertility rates, birth counts, a doe's full breeding record — is built from the **permanent logs**, never from the live `Breeding` row. Counting off the reused row would lose every past cycle. (A recent fix moved the fertility tables onto the logs for exactly this reason.)

8 Data-duplication analysis

The direct answer to "is there any data duplication?": **yes — but almost all of it is intentional and necessary**, and the schema has explicit machinery to stop the *unintentional* kind. Here is the breakdown.

8.1 Intentional duplication (by design, correct)

Where	What's duplicated	Why it's right
Archive logs vs Breeding/Litter	Each log snapshots `doeId`, `buckId`, `matingDate` (and counts) that also live on the live row.	The live row is overwritten on the next cycle; without the copy the event would vanish. This is a deliberate *snapshot*, not redundant storage.
KindlingLog / WeaningLog born-counts	`bornAlive`, `bornDead`, `weaned`, `weaningWeightGrams` are mirrored from the Litter.	One-way mirror while the litter is the doe's current cycle, then frozen. Keeps the log self-contained after the Litter moves on.
KitStockMovement ↔ Transaction	A kit *sale* writes both a stock movement and a matching ledger Transaction (linked 1:1).	The stock ledger and the money ledger are different views; the link keeps them consistent, the `@@unique` stops a double-mirror.
KitStockMovement ↔ Rabbit	A *retained* kit writes a stock movement and promotes a Rabbit (linked 1:1, cascade).	Deducts from the batch pool while creating the individual animal; deleting the rabbit undoes the deduction.
Pedigree vs Litter	A kit's parents live on `sireId/damId` even though the litter also implies them.	The FK is authoritative so *acquired* rabbits (known parents, never bred here) work too.
Mobile SQLite mirror	The entire working dataset is duplicated onto every device.	That's what makes the app work offline. The server stays the single source of truth.

8.2 How *unintentional* duplication is prevented

- **Client-generated IDs.** Offline "create" operations mint the row's ID on-device, so the optimistic local row and the server row share it — dedup by ID, not by guessing.
- `INSERT OR REPLACE` **by ID.** A re-pulled row overwrites its local twin instead of adding a copy.
- `clientOpId` **idempotency.** A resent push batch skips already-applied operations.
- **Composite** `@@unique` **keys.** One tag per sex per farm; one breed name per farm; one litter per breeding.
- **Day-keyed cycle stitching.** A doe is mated at most once per day, so history views group a cycle's stages by calendar day — a fractional timestamp drift can't split one mating into two rows.
- **SyncTombstones.** Deletes propagate, so a device can't keep a phantom copy of a removed row.

8.3 Duplication bugs recently found & fixed

These were real defects in test data; each now has a structural guard so it can't recur.

Symptom	Root cause	Fix

A mating showing **twice** in a doe's سجل التزاوج	History keyed on the raw timestamp; snapshots drifted by a fraction of a day and split one cycle into two rows.	All history stitchers now key by calendar day; MatingLog creation is idempotent per (doe, mating-day).
"2 = عدد مرات التلقيح" for a single mating	MatingLog had no replay-dedup, so an op replay could append a second archive row.	An idempotency guard checks for an existing (doeId, matingDate) row before inserting.
Duplicate pregnancy-test rows	Local optimistic row and server row were matched by drifting `matingDate`.	Both now share a client-generated ID; sync dedups by ID (`INSERT OR REPLACE`).
سجل التلقيح date ≠ سجل الجس date after an edit	Editing a mating date updated the live row + later snapshots but not the append-only MatingLog.	The edit paths (`updateBreedingOp` & `setMatingDateOp`) now carry the correction into the existing MatingLog row.

Watch-items (not bugs, but worth knowing)

- **Snapshot mirrors can go stale after edits.** The born-count / date mirrors from Litter into the logs are one-way and best-effort. If a source value is corrected long after the cycle closed, the historical mirror won't move unless an edit path explicitly propagates it (as the mating-date fix now does).
- **Litter joins are day-level, best-effort.** Views that attach litter counts to a log join on doe + kindling-day, because a Litter can only point at the current Breeding row. Correct in practice given "one cycle per doe per day", but it's a soft join, not a hard FK.
- **Existing mis-dated rows aren't retro-fixed.** The fixes above prevent recurrence; already-drifted test rows are cleared by the pending data reset, not corrected in place.

9 Allowed value sets (enum-like fields)

These are stored as plain strings (validated in the app, not native DB enums) so the SQLite mirror and Postgres stay identical. Authoritative list:

Field	Allowed values
Rabbit.sex	`buck` · `doe` · `unknown`
Rabbit.status	`active` · `sold` · `culled` · `deceased` · `reference` · `resting`
Rabbit.doeState	`empty` · `bred` · `pregnant` · `nursing` · `nursing_bred` · `nursing_pregnant` · `excluded`
Rabbit.origin	`external` · `farm`
Breeding.outcome	`pending` · `successful` · `failed` · `not_pregnant`
Breeding.pregnancyTestResult	`pending` · `positive` · `negative`
HealthRecord.type	`vaccination` · `treatment` · `illness` · `deworming` · `checkup`
Transaction.type	`income` · `expense`
Transaction.category	`sale` · `purchase` · `feed` · `vet` · `equipment` · `other`
KitStockMovement.type	`sale` · `death` · `retained`
FarmMember.role	`owner` · `worker`
Settings.weightUnit	`kg` · `lb_oz`

Note: a couple of Prisma schema comments list shorter value sets (e.g. doeState, status) than the application actually uses — the app's enum file above is the real source of truth. Aligning the comments is a tidy-up, not a data issue.

10 Findings & recommendations

> **Overall assessment**
>
> The data model is well-structured and deliberately designed for an offline-first, multi-tenant, history-preserving application. The duplication that exists is overwhelmingly **intentional snapshotting** required by the reused-row design, and the schema carries real machinery (client IDs, idempotency keys, unique constraints, tombstones, day-keying) to prevent the accidental kind.

1. **Keep counts sourced from the permanent logs**, never the live Breeding/Litter rows — this is already the rule; hold the line on it in any new report.
2. **Propagate corrections into snapshots.** Whenever an edit changes a value that a log snapshotted (date, buck, counts), the edit path should carry it into the matching log row — the pattern the recent mating-date fix established. Consider a shared helper so every future edit path does this uniformly.
3. **Align the small schema/comment drifts**: mobile `settings_cache` defaults vs server defaults; the abbreviated enum comments in the Prisma schema. Cosmetic, but they mislead a cold reader.
4. **Consider a light integrity check** (a periodic query) that flags any log row whose snapshot disagrees with a still-live source, to catch stale mirrors early rather than at report time.
5. **The pending data reset is the right move** for the current test data — it clears the already-drifted rows and re-bootstraps every device against the corrected server. Run it only after every device has the latest build.
